L3-INFO Algorithmique

Quick 2 – 5 novembre 2025 – Durée 1 h

Une feuille A4 manuscrite recto-verso autorisée, ainsi que les dictionnaires bilingues.
Les dispositifs électroniques sont interdits, à l'exception des montres non connectées.
Le barème est indicatif.

Vous êtes vivement encouragés à illustrer vos recherches et vos réponses par des schémas.

Un tableau de *Young* est un tableau bidimensionnel de taille $m \times n$, tel que tous les éléments d'une même ligne sont triés de gauche à droite et tous ceux d'une même colonne sont triés de haut en bas. Si on a moins de $m \times n$ valeurs à stocker, on complète en remplissant les cellules « vides » par une valeur spéciale $+\infty$ (supérieure à toutes les autres).

Ainsi, un tableau possible pour stocker les valeurs $\{9, 16, 3, 2, 4, 8, 5, 14, 12\}$ est le tableau Y suivant :

$Y =$

	1	$\cdots$		n
1	2	8	12	14
	3	9	16	$+\infty$
$\vdots$	4	$+\infty$	$+\infty$	$+\infty$
	5	$+\infty$	$+\infty$	$+\infty$
m	$+\infty$	$+\infty$	$+\infty$	$+\infty$

$\leqslant$ (→) ; $\geqslant$ (↓)

On indexe les lignes de Y de 1 à m, et ses colonnes de 1 à n.

1. **Minimum**

(1 pt) (a) Où se trouve le plus petit élément d'un tableau de Young non vide ?

(1 pt) (b) Que peut-on dire de $Y[m][n]$ si le tableau n'est pas plein ?

> **Solution:**
> (a) Dans la cellule de coordonnées $(1, 1)$.
> (b) Elle contient forcément $+\infty$ (car cette cellule doit être supérieure à toutes les autres).

2. **Exemple d'insertion**

On insère le nombre 6 dans la cellule à la ligne 2 et à la colonne 4 du tableau de l'exemple.

(1 pt) Montrez comment on peut le « remonter » de proche en proche jusqu'à sa place en ne faisant que des échanges avec des cellules voisines, de façon à obtenir un tableau de Young.

Solution: On l'échange successivement avec $16, 12, 8$. Il n'y a pas d'autre chemin possible, sinon on casse la propriété de Young.

3. **Percolation**

On suppose que l'on remplace la valeur de la cellule à la ligne i et à la colonne j par une valeur $v < Y[i][j]$.

(3 pts) (a) Écrivez une fonction `Remonte(i,j,v,Y)` qui « remonte » le nouvel élément placé dans la cellule $Y[i][j]$ pour le mettre à sa place.

(2 pts) (b) Donnez et justifiez que le coût de cette fonction en nombre de comparaisons avec des élements de Y.

Solution:

(a) — Expression à haut niveau :

$Y[i][j] := v$

tant que *($i > 1$ ou $j > 1$) et $Y[i][j] <$ au moins un de ses voisins* **faire**

  Échanger $Y[i][j]$ avec le plus grand de ses voisins du dessus ou de gauche

  Continuer avec les indices i et j de ce voisin

On ne compare qu'avec le voisin de gauche $Y[i][j-1]$ et le voisin du dessus $Y[i-1][j]$, ce sont les seuls qui peuvent être mal placés par rapport à v.

Par contre il faut « gérer les bords », c'est-à-dire éviter les indices $i-1$ ou $j-1$ nuls.

— Expression détaillée avec les indices :

tant que *($i > 1$ et $v < Y[i-1][j]$) ou ($j > 1$ et $v < Y[i][j-1]$)* **faire**

  si $i == 1$ **alors**

    $Y[i][j] := Y[i][j-1]$

    $j := j - 1$

  sinon si $j == 1$ **alors**

    $Y[i][j] := Y[i-1][j]$

    $i := i - 1$

  sinon si $Y[i-1][j] > Y[i][j-1]$ **alors**

    $Y[i][j] := Y[i-1][j]$

    $i := i - 1$

  sinon

    $Y[i][j] := Y[i][j-1]$

    $j := j - 1$

$Y[i][j] := v$

Pour simplifier l'écriture de l'algorithme on pourrait remplir les lignes et colonnes d'indice 0 avec des valeurs $-\infty$.

(b) Elle est en $\mathcal{O}(n+m)$.

Chaque itération diminue i ou j de 1, or initialement $i \leqslant m$ et $j \leqslant n$ et ces deux indices ne peuvent descendre en-dessous de 0, il est donc clair que cet algorithme ne peut effectuer plus de $n+m$ itérations.

Chaque itération donne lieu au pire à 3 comparaisons avec des éléments de Y, le coût total est donc $< 3(n+m)$.

4. **Recherche**

Voici un algorithme qui recherche si une valeur x est présente ou non dans un tableau de Young :

```
RECHERCHE_YOUNG(x, Y)
  Données : Une valeur x et un tableau de Young Y de taille m × n
  Résultat : Les coordonnées d'une cellule de Y contenant x,
             ou bien (0, 0) si x n'est présent nulle part dans Y

1  i := m
2  j := 1
3  tant que i > 0 et j ⩽ n faire
4      si Y[i][j] < x alors
5          j := j + 1
6      sinon si Y[i][j] > x alors
7          i := i − 1
8      sinon
9          renvoyer (i, j)
10 renvoyer (0, 0)
```

(2 pts) (a) Justifiez que cet algorithme se termine à l'aide d'un variant.

> **Solution:** Le variant $i + n - j$ convient : il est bien entier, positif (car $i > 0$ et $j \leqslant n$), et strictement décroissant (il diminue de 1 à chaque itération).
> Attention, $i - j$ n'est pas toujours positif ! Si on veut l'utiliser il faut au moins justifier qu'il est minoré par $-n$.

(b) Il est assez clair que quand l'algorithme renvoie un couple $(i, j) \neq (0, 0)$ ce résultat respecte la spécification.

Il nous reste donc à démontrer la **postcondition** suivante :

« Quand l'algorithme renvoie $(0, 0)$ aucune cellule de Y ne contient x. »

(1 pt) i. Que peut-on affirmer à propos de la boucle **tant que** lorsque l'algorithme a renvoyé $(0, 0)$?

(1 pt) ii. Représentez sous forme de schéma un invariant vérifié par la boucle **tant que** de notre algorithme (juste avant la ligne 4), à propos de la valeur x et du contenu de certaines cellules de Y.

En sortie de boucle, votre invariant devrait bien entendu permettre de démontrer la postcondition donnée ci-dessus.

(2 pts) iii. Exprimez également cet invariant sous la forme d'une phrase en français, en faisant référence explicitement à des indices du tableau.

(1 pt) iv. Justifiez que cet invariant est vérifié après la ligne 2 de l'algorithme.

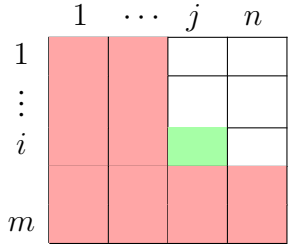
(3 pts) v. Démontrez que la boucle **tant que** maintient effectivement cet invariant.

(2 pts) vi. Enfin, à l'aide de votre invariant en sortie de boucle, démontrez la postcondition énoncée ci-dessus.

> **Solution:**

i. Si l'algorithme renvoie $(0, 0)$ c'est qu'on est sorti de la boucle (sans rencontrer le **renvoyer**), donc que $i == 0$ ou que $j > n$.

ii. Sur le schéma :
— la cellule colorée en vert est le (i, j) courant ;
— tous les éléments colorés en rouge sont différents de x.



iii. Dans Y, toutes les cellules de coordonnées (a, b) avec $a > i$ ou $b < j$ contiennent des éléments différents de x.
Ou bien : $\forall 1 \leqslant a \leqslant m,\ \forall 1 \leqslant b \leqslant n,$ si $a > i$ ou $b < j$, alors $Y[a][b] \neq x$.
Ou encore, de façon équivalente, si $Y[a][b] == x$ alors $a \leqslant i$ et $b \geqslant j$.

iv. Comme $i = m$ et $j = 1$, à la ligne 2 il n'y a aucune coordonnée (a, b) valide dans Y telle que $a > i$ ou $b < j$.

v. On suppose l'invariant vérifié en début d'itération.
Le cas $Y[i][j] == x$ fait sortir de la boucle avec un résultat visiblement correct ; pour la propagation de l'invariant il n'y a que deux cas à considérer :
— Si $Y[i][j] < x$ alors on incrémente j. Les nouvelles cellules concernées par la « zone rouge » sont les $Y[a][j]$ (pour l'ancien j) avec $a \leqslant i$. Mais comme elles sont au dessus de $Y[i][j]$ elles sont plus petites, et a fortiori elles sont $< x$.
— Si $Y[i][j] > x$ alors on décrémente i. Les nouvelles cellules concernées par la « zone rouge » sont les $Y[i][b]$ (pour l'ancien i) avec $b \geqslant j$. Mais comme elles sont à droite de $Y[i][j]$ elles sont plus grandes, et a fortiori elles sont $> x$.

vi. En sortie de boucle $i = 0$ ou $j > n$.
Dans chacun des deux cas, toutes les coordonnées (a, b) valides dans Y vérifient $a > i$ (respectivement $b < j$), elles contiennent donc toutes des éléments différents de x.